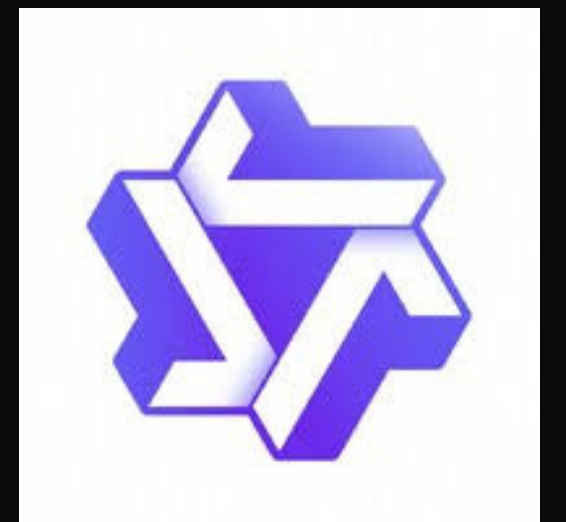
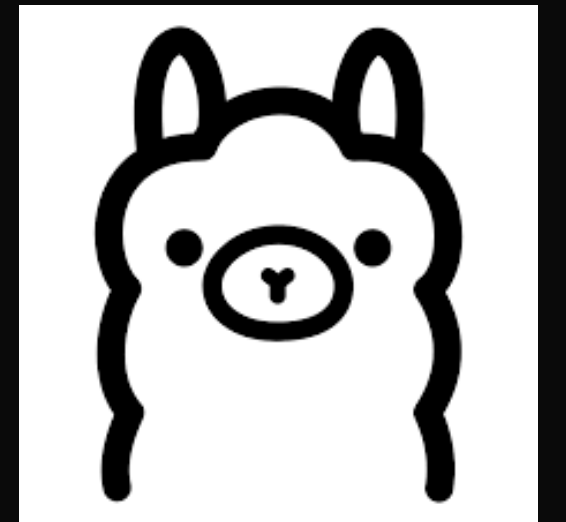
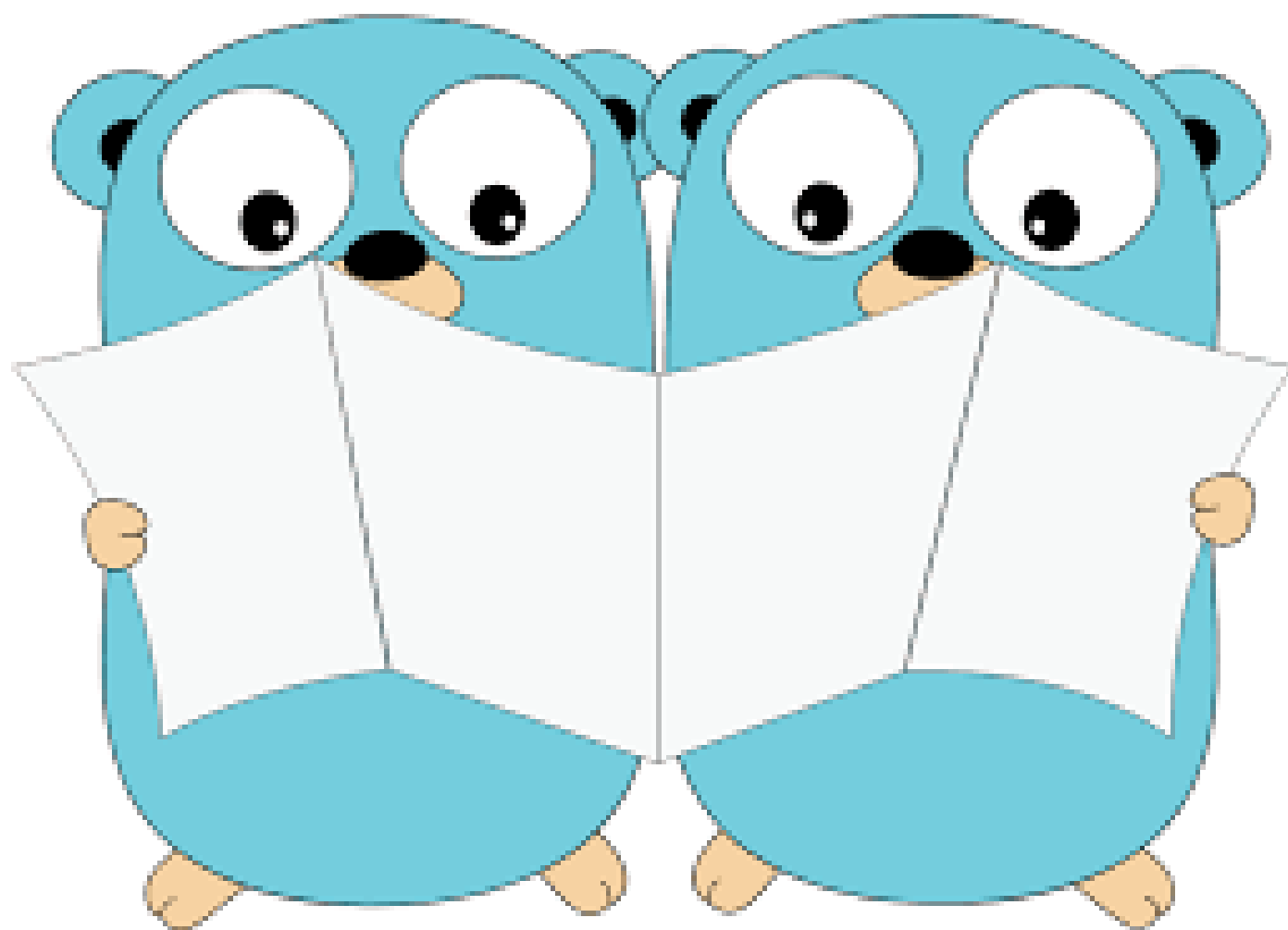


Making OpenClaw like Agent in



Contents



1 What is agent

2 Architecture

2 Agentcore and Planning

3 Memory and context

4 Skills And Tools

5 Channels

6 Personality

7 Github Link

What is an agent

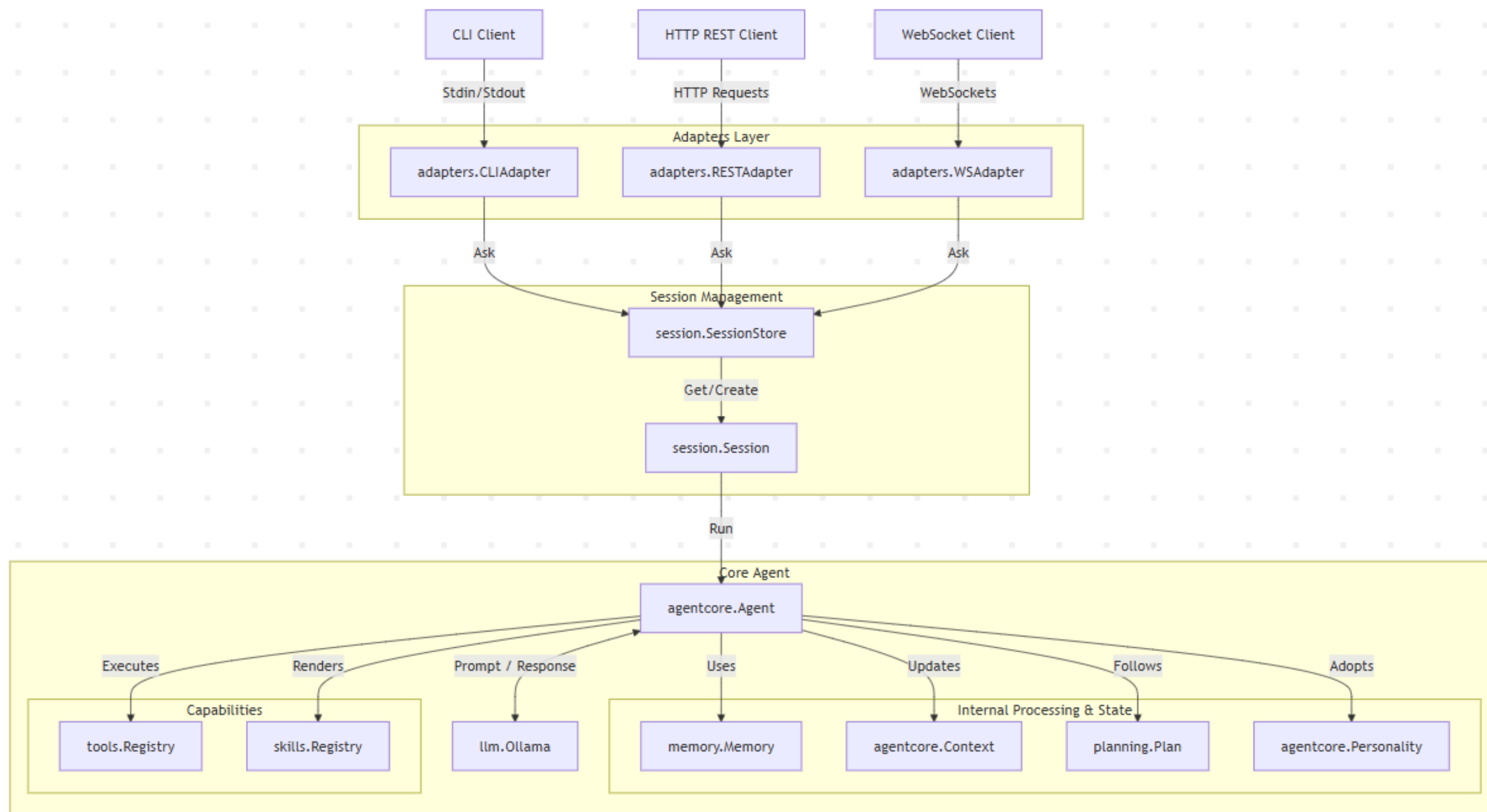
A minimal claud like autonomous agent in Go with:

- Tools (calculator, shell, etc.)
- Works locally with ollama
- Skills (loadable from directory)
- Memory (short-term + commit log)
- Personality (loaded from markdown)
- Planning + step execution loop

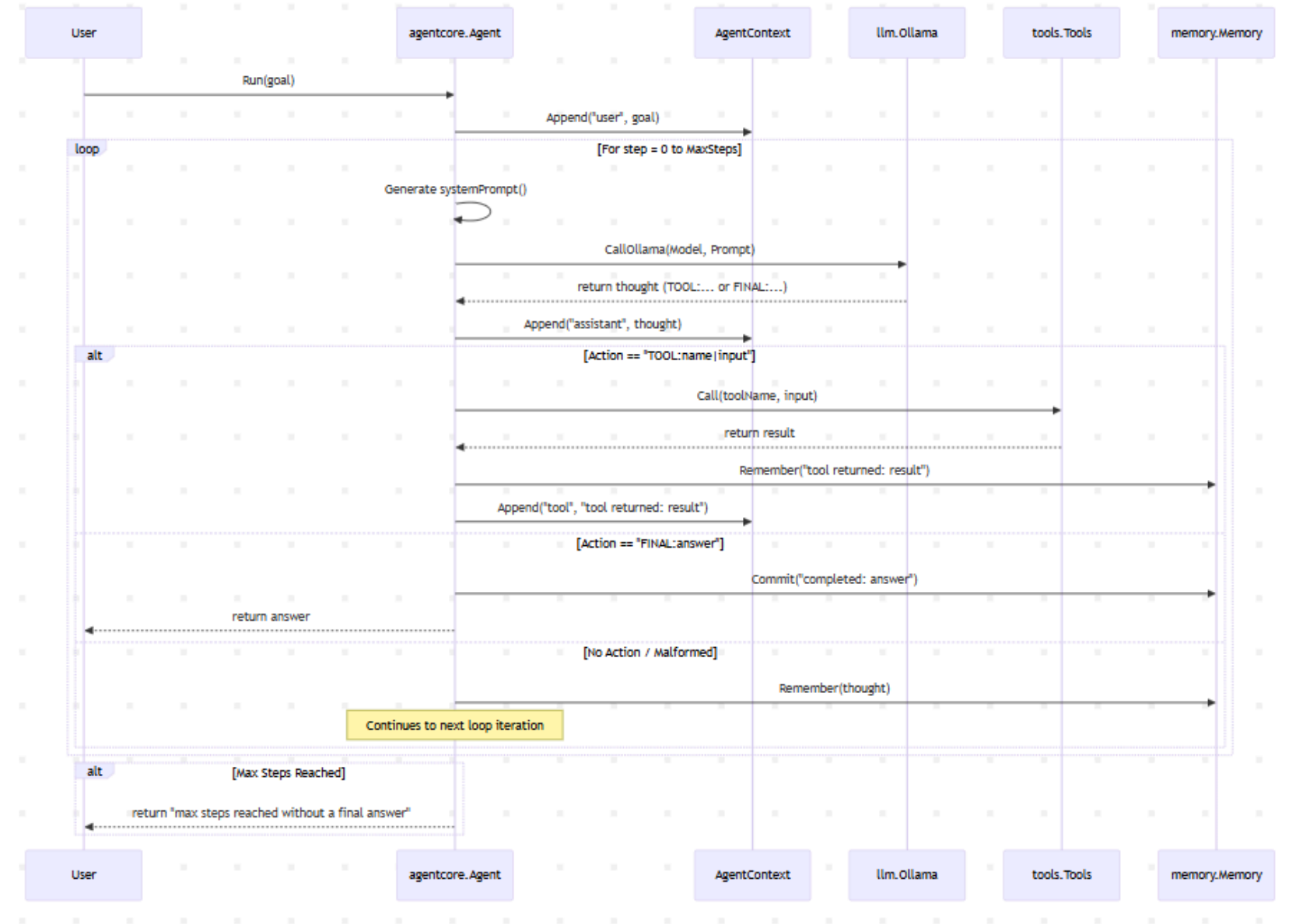
Agent can connect via multiple channels and take query as goal and breakdown into tasks and use tools and skills to achieve the answer for the user. Ollama is used for llm. other apis can be extended.

It is a prototype with a many things to implement.

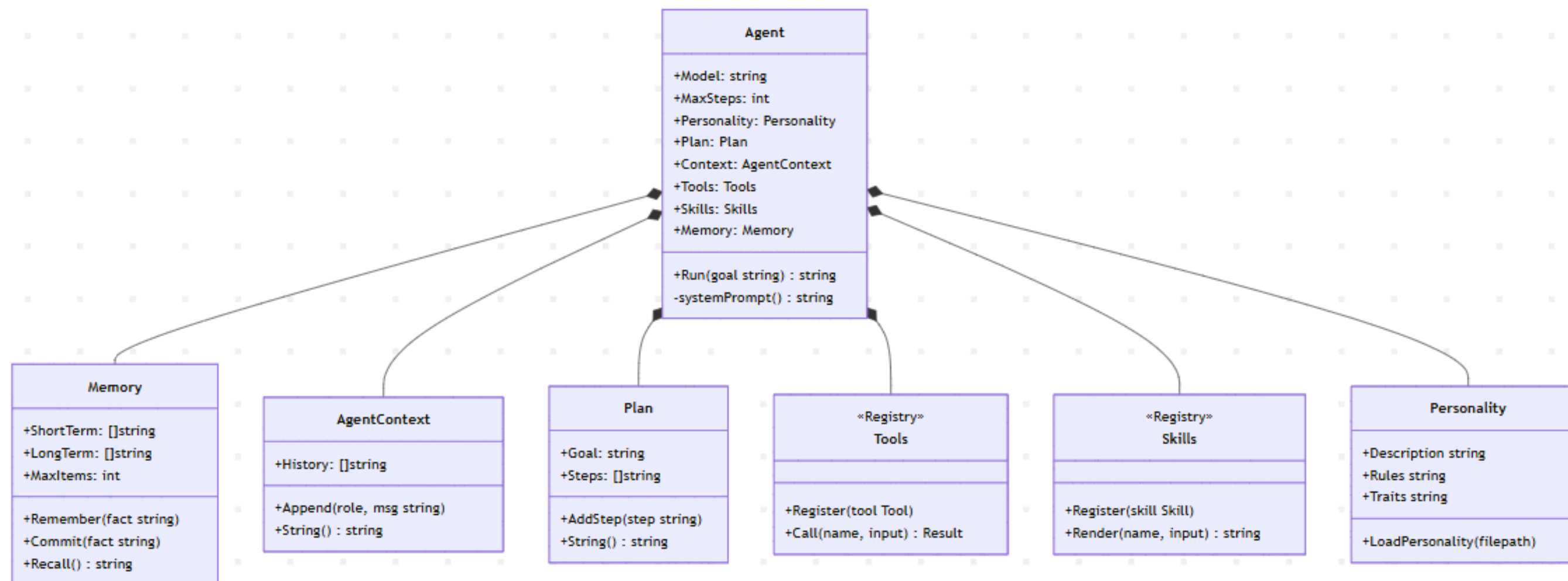
Architecture



Flow



Classes



Agent Core

Agent Core is the component that binds together the components such as memory, planning, personality, skills etc.

it has a dedicated prompt for the description and the Run function is responsible to taking query and start the agent process of goal to result. session is wrapped around it and channel adapters are used to connect and get results.

```
1 // Agent
2 type Agent struct {
3     Model      string
4     MaxSteps   int
5     Personality *Personality
6     Plan       planning.Plan
7     Context    AgentContext
8     Tools      tools.Tools
9     Skills     skills.Skills
10    Memory     *memory.Memory
11 }
12
13 func NewAgent(model string, maxSteps int, personality *Personality) *Agent {
14
15     return &Agent{
16         Personality: personality,
17         Model:       model,
18         MaxSteps:    maxSteps,
19         Tools:       make(tools.Tools),
20         Skills:      make(skills.Skills),
21         Memory:      memory.NewMemory(10),
22     }
23 }
24
25
26 func (a *Agent) systemPrompt() string
27
28
29 func (a *Agent) Run(goal string) string
```


planning

agent planning is generation a list of steps

with a state wether the steps are completed to achieve a goal asked in query.

agent genreates step for goal then executes them to get the answer.

it may also include relevelt tool calling or skill reference to achieve the goal.

goal have fixeed steps under which the answer should be provided , if not then it reponds with maximum steps/tries reached

```
1  package planning
2
3  import (
4      "fmt"
5      "strings"
6  )
7
8  // plan
9  type Plan struct {
10     Goal  string
11     Steps []string
12 }
13
14 func (p *Plan) AddStep(step string) {
15     p.Steps = append(p.Steps, fmt.Sprintf("step %d: %s", len(p.Steps), step))
16 }
17
18 func (p *Plan) String() string {
19     return fmt.Sprintf("Goal: %s\n%s", p.Goal, strings.Join(p.Steps, "\n"))
20 }
21
```


memory

agent has 2 types of memory for session

- short term : mostly for session context that automatically compresses when its length increases
- long term:

when things need persistence it goes to long term

- session: when communication is made via different channels each has a session key

```
1 package memory
2
3 import (
4     "strings"
5 )
6
7 // memory
8
9 type Memory struct {
10     ShortTerm []string
11     LongTerm  []string
12     MaxItems  int
13 }
14
15 func NewMemory(maxItems int) *Memory {
16     return &Memory{MaxItems: maxItems}
17 }
18
19 func (m *Memory) Remember(fact string) {
20     m.ShortTerm = append(m.ShortTerm, fact)
21     if len(m.ShortTerm) > m.MaxItems {
22         m.ShortTerm = m.ShortTerm[1:]
23     }
24 }
25
26 func (m *Memory) Commit(fact string) {
27     m.LongTerm = append(m.LongTerm, fact)
28 }
29
30 func (m *Memory) Recall() string {
31     all := append(m.LongTerm, m.ShortTerm...)
32     return strings.Join(all, "\n")
33 }
34
```

context

agentic context is a joint text with various things like skill details, tools description, short term memory, agent prompt, personality etc.

this is used by agent to understand the query and work according and take action to provide result

```
package agentcore
```

```
import (  
    "fmt"  
    "strings"  
)
```

```
// context
```

```
type AgentContext struct {  
    History []string  
}
```

```
func (c *AgentContext) Append(role, msg string) {  
    c.History = append(c.History, fmt.Sprintf("[%s] %s", role, msg))  
}
```

```
func (c *AgentContext) String() string {  
    return strings.Join(c.History, "\n")  
}
```

skills

skills are the latest addition to agentic workflow

skills are the md documents that contains a reference on what to do and how to do it with example commands like

- git operations push pull commit etc
- specific calculation
- based on task its domain knowledge as a tool

they reduce the contextlength and provide a more clear and predictable answer and can be expanded vary easily as they are md files

```
1  # Skill: command-run (PowerShell)
2
3  ## Description
4
5  Executes user requests by breaking them into **single, safe PowerShell commands**, running o
6
7  This skill is designed for controlled, step-by-step shell execution.
8
9  ---
10
11 ## Behavior Rules
12
13 1. Interpret the user's request.
14 2. Break it into **atomic PowerShell commands** (one action per step).
15 3. Execute each command sequentially.
16 4. Show:
17
18     * The command being executed
19     * The output
20 5. Do not combine multiple operations into one command unless necessary.
21
22 ---
23
24 ## Execution Format
25
26 Each step must follow:
27
28 ```id="9d9h2a"
29 Command:
30 <PowerShell command>
31
32 Output:
33 <result>
34 ```
35
36 ---
37
38 ## Safety Constraints
39
```

tools

tools are the functions with names and description that the agent can refer and call based on the task type

the tools are loaded via tool registry ,
each tool has a specific input and output type
and they can be expanded as per the requirements.

the agent gets the result from tool then
formats it as required answer

```
1 package tools
2
3 import "fmt"
4
5 func calculate(input string) string {
6     var a, b float64
7     var op string
8
9     _, err := fmt.Sscanf(input, "%f %s %f", &a, &op, &b)
10    if err != nil {
11        return "error: expected format 'a op b', got: " + input
12    }
13
14    switch op {
15    case "+":
16        return fmt.Sprintf("%g", a+b)
17    case "-":
18        return fmt.Sprintf("%g", a-b)
19    case "*":
20        return fmt.Sprintf("%g", a*b)
21    case "/":
22        if b == 0 {
23            return "error: division by zero"
24        }
25        return fmt.Sprintf("%g", a/b)
26    default:
27        return "error: unknown operator: " + op
28    }
29 }
30
31 func CalculatorTool() Tool {
32     return Tool{
33         Name: "calculator",
34         Description: `performs arithmetic on two numbers. input format:
35                    'a op b' where op is +, -, *, /. example: '12.5 * 3'`,
36         Fn: calculate,
37     }
38 }
39
```


channels

the agent can communicate via multiple channels .
Adapter pattern is used to keep the agent and channels separate so new channels can be added without updating the agent code.

at present it has cli , rest ,ws channel
new channel like telegram , slack , whatsapp can be added by implementing adapter pattern

```
1 package adapters
2
3 import (
4     "agent-in-go/pkg/session"
5     "bufio"
6     "fmt"
7     "os"
8     "strings"
9 )
10
11 type CLIAdapter struct {
12     store *session.SessionStore
13 }
14
15 func NewCLIAdapter(store *session.SessionStore) *CLIAdapter {
16     return &CLIAdapter{store: store}
17 }
18
19 func (a *CLIAdapter) Name() string { return "CLI" }
20
21 func (a *CLIAdapter) Start() error {
22     scanner := bufio.NewScanner(os.Stdin)
23     fmt.Println("[CLI] agent 451 ready. ctrl+c to exit")
24     for {
25         fmt.Print("\n> ")
26         if !scanner.Scan() {
27             break
28         }
29         input := strings.TrimSpace(scanner.Text())
30         if input == "" {
31             continue
32         } else if input == "bye" {
33             fmt.Println("good bye")
34             break
35         }
36         fmt.Println("answer:", a.store.Ask("cli", input))
37     }
38     return nil
39 }
40
```

Personality

Agent has a personality that is described in a markdown file. It can be customized as per requirements to become useful for the user.

it has name, personality traits and rules that the user can configure based on his requirements



Personality

name: Joy

Description

You are an assistant who talks like a cool person.

You use casual slang, contractions, and a warm tone.

Traits

- Friendly
- Playful
- Casual speech

Rules

- Do not break character
- Keep responses concise
- Avoid overdoing slang

Github Link

Github Link :-

<https://github.com/subpxl/GopherAgent>

-

github.com/subpxl

Shubham Panchal

